
Structured Rule Variation as Pressure for Modularization in Transformers

E. Spencer Shelton
University of Chicago
spencershelton@uchicago.edu

Abstract

Transformers often represent behavior in densely shared parameter spaces, making it difficult to target specific aspects of a model after training. This paper studies whether structured variation over related rule systems can create training-time pressure for modularization in transformers. Chess provides a controlled setting for studying this; its rules are explicit and programmatically modifiable, and synthetic data can be generated with precisely defined, targeted rule changes. We train transformer models on next-move prediction across both standard chess games and games drawn from a structured family of per-piece movement-rule variants, with the models including piece-specific module slots at fixed positions in the architecture associated with piece-rule variants. The central hypothesis is that exposure to many related per-piece rule systems, paired with architectural constraints that limit shared computation and select rule-specific modular capacity according to the current ruleset, can encourage rule-specific behavior to localize in modular components. We evaluate this hypothesis using rule-conditional performance, targeted ablations, module swaps, and comparisons among modular architectures. As a stronger downstream test, we examine whether held-out movement rules can be incorporated by training only newly introduced module slots, and measure both gains on rule-specific moves and interference on unchanged parts of play. More broadly, this project tests whether rule-family training can induce functional modularity in transformers, with possible implications for lower-interference adaptation in highly structured, rule-governed domains such as programming languages.

1 Introduction

Transformers can be trained as dense networks where examples from many contexts update a shared parameter space, and downstream behaviors emerge from interactions among many layers and attention heads. This creates a practical difficulty that after training, it can be unclear how one might target a specific behavior without affecting many unrelated behaviors. This paper asks whether the training distribution itself can be structured to pressure a transformer toward reusable modular components.

This question is studied in the context of chess. Chess is useful here not because chess play is the final application, but as it gives a controlled rule-governed environment where rules are explicit, legal move generation can be checked programmatically, and synthetic games can be generated under precisely specified rule changes. For this paper a family of chess-like games was constructed in which individual piece movement rules are changed. The model is trained for next-move prediction over standard chess and over games sampled from combinations of per-piece rule variants using a specialized vocabulary designed to represent the games. The model is not given any indicator in the input of what ruleset is active, instead the training scaffold selects module slots associated with the active per-piece rule variants.

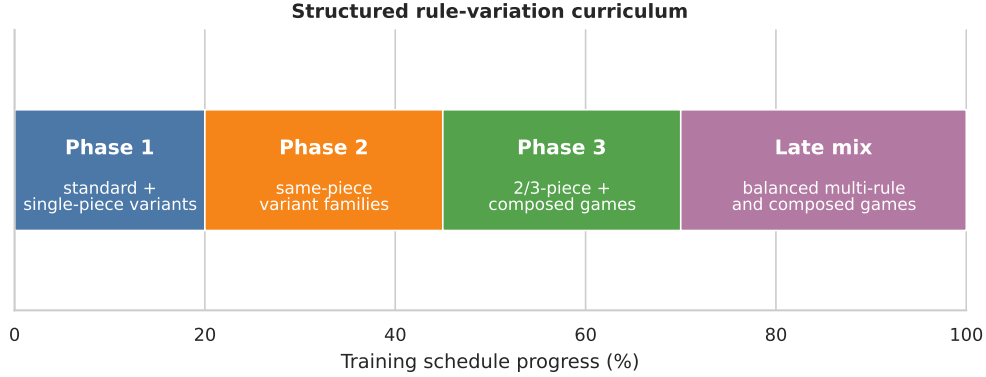


Figure 1: Training curriculum. The data schedule moves from standard and single-piece variants toward multi-rule and composed games, increasing pressure to represent rule-dependent behavior.

The motivating hypothesis is that structured rule variation can make it useful for the network to store rule-specific behavior in corresponding module slots. If this occurs, then changing or adding a rule should be possible by changing or training a small part of the model, with limited interference elsewhere. This paper asks whether module slots carry functionally relevant information, whether wrong module selections damage the relevant behavior, and whether held-out rules can be learned by training newly introduced module slots.

The contributions this paper attempts to make are:

- Introducing a controlled rule-family training setup based on per-piece chess movement variants.
- Comparing three modular transformer variants: a two-stage sequence model, a single-stage sequence model, and a position-only single-stage model.
- Evaluating modularity using held-out rule adaptation, wrong-module stress tests, rule-conditional performance, and regression on unchanged parts of play.
- Finding evidence for partial functional localization: held-out modules are necessary for unique held-out move geometry, but unpolished modules can interfere with unchanged context moves. A short polish pass reduces this interference substantially.

2 Setup: structured chess rule variation

Let a ruleset be represented by a vector of six piece-rule variant identifiers, one for each piece type. The standard ruleset has all identifiers set to zero and a nonzero entry changes the movement rule for one piece type while leaving the rest of the game structure intact. Training games include standard chess, single-piece variants, multi-piece variants, and composed games that splice games at shared positions with composition exposing the model to contexts in which the same board position can appear under different rule histories and rule combinations.

Two movement rules are held out from the main rule-family training and used for adaptation tests: a king rule allowing a two-square diagonal move (K5), and a knight rule allowing a two-by-four leap (N5). These rules create move geometries absent from the seen rule families and allows us to define what should be a reasonably strong diagnostic: on positions where the target move is legal only under the held-out rule geometry, performance without the corresponding held-out module should be poor, while performance with a trained held-out module should improve.

3 Models

All models are decoder-style transformers trained for next-token next-move prediction. Moves are tokenized into structured components: piece, source square, target square, and termination or promotion tokens. We study three model families (Figure 2).

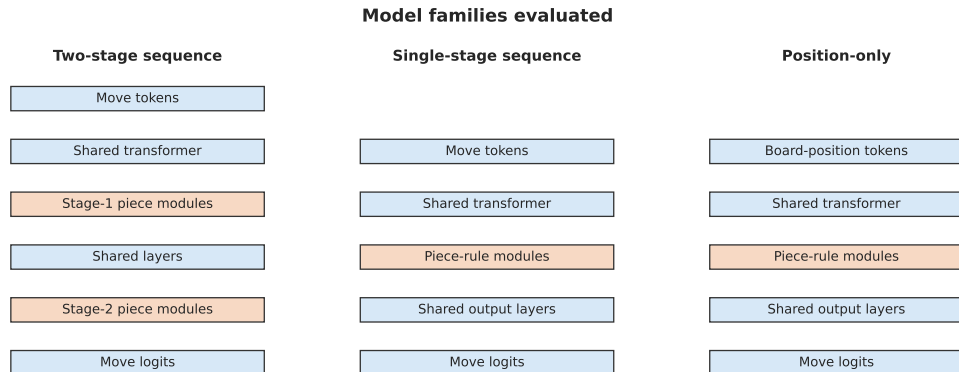


Figure 2: Model families evaluated. The sequence models consume move histories, while the position-only model consumes a board-position prompt. Orange blocks indicate piece-rule module slots.

Two-stage sequence model. The first architecture processes move-sequence context and inserts two sets of piece-rule module slots. The first stage is intended to propagate rule-relevant information into the representation and the second stage is closer to the output and is intended to influence move decisions. This design tests whether a deeper modular pathway improves rule-localized behavior.

Single-stage sequence model. The second architecture removes the second modular stage and keeps one set of piece-rule module slots. This reduces the number of modular mechanisms and tests whether a simpler design is easier to train and interpret.

Single-stage position model. The third architecture receives a board-position prompt rather than a move-sequence prefix. This tests whether sequence history lets the model infer rule context in a way that weakens dependence on modules. The position-only model is still trained to predict the next move, but its input is the current board state plus selected module slots rather than the full move history.

In all modular models, the selected module slots are determined by the active piece-rule identifiers. This is not learned dynamic routing; the active rule vector determines which modular capacity is loaded or selected.

4 Training

The training objective is next-token cross-entropy over legal game continuations. Standard chess remains a substantial part of the distribution so that the model retains ordinary chess competence, while variant data provides rule-specific pressure. The high-level curriculum is shown in Figure 1 and we also use pressures intended to discourage the shared layers from absorbing all rule-specific behavior in using lower effective learning rates for shared layers, module-specific learning-rate multipliers, and later training phases in which module-only and shared-only updates are separated.

After the main rule-family training, we train held-out K5 and N5 module slots while freezing shared layers and non-held-out modules. The held-out training examples come from games where the corresponding held-out rule is active and then running a short polish pass with the full model loaded but with shared layers and other modules still frozen. This pass is intended to reduce interference on other moves that occur in games where held-out modules are active, not merely to improve the held-out piece moves themselves.

The final held-out evaluations use matched samples of 3,840 examples across standard, seen-variant, held-out active-piece, held-out unique-geometry, and held-out context subsets.

5 Evaluation

We use four complementary evaluations.

Table 1: Polished held-out K5+N5 modules compared with base normal modules on the matched full evaluation. Values are base $\rightarrow$ polished.

Model	Overall top-4	Unique geometry loss	Context loss
Single-stage sequence	0.918 $\rightarrow$ 0.932	1.660 $\rightarrow$ 0.798	0.855 $\rightarrow$ 0.767
Two-stage sequence	0.940 $\rightarrow$ 0.952	1.816 $\rightarrow$ 0.764	0.708 $\rightarrow$ 0.700
Single-stage position	0.801 $\rightarrow$ 0.838	3.009 $\rightarrow$ 0.846	1.431 $\rightarrow$ 0.952

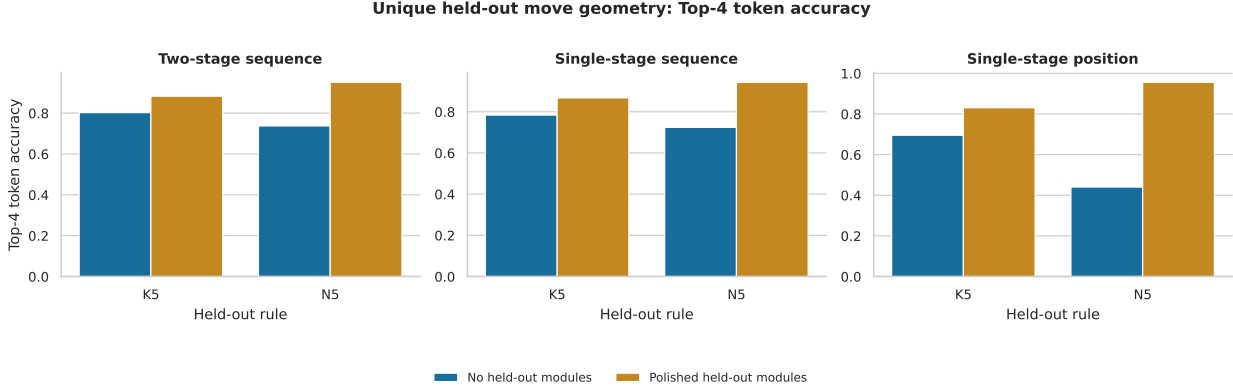


Figure 3: Top-4 token accuracy on K5 and N5 unique move geometries. Polished held-out modules improve unique-rule behavior relative to the base model without held-out modules.

Rule-conditional next-move performance. We measure exact move accuracy, token accuracy, top-4 token accuracy, loss, and component accuracies. Top-4 token accuracy is useful as move tokens can be decomposed into piece and geometry components, and exact-move accuracy is strict.

Unique held-out geometry. We isolate K5 and N5 moves whose source-to-target geometry is absent from seen rules as this should directly test whether the trained held-out module contributes behavior that the base model could not easily express using seen rule modules.

Context interference. Held-out rules are active for an entire game, but not every move in such a game is made by the held-out piece. We therefore evaluate moves by other pieces in held-out-rule games so that if loading a held-out module damages these moves, the adaptation is not clean even if rule-specific moves improve.

Wrong-module stress tests. We evaluate the same held-out unique moves while replacing the moving piece’s held-out module selection with the standard module or seen variant modules testing whether performance depends on the correct held-out module slot rather than on information stored entirely in shared layers.

6 Results

6.1 Held-out modules learn rule-specific behavior

The held-out modules appears to improve performance on held-out rule settings. In the full polished evaluation, moving from the base normal modules to polished K5+N5 modules improves overall loss and top-4 accuracy across all three model families with the strongest gains appearing on held-out unique geometry and held-out active-piece moves (Table 1). For the position model, unique-geometry loss falls from 3.01 to 0.85 and top-4 accuracy rises from 0.57 to 0.89. The sequence models also improve, though exact accuracy remains more modest than the others.

6.2 Polishing reduces context interference

Training held-out modules only on the held-out piece improves rule-specific moves but seems to interfere with other moves in games where the held-out rules are active, but the matched polished-vs-

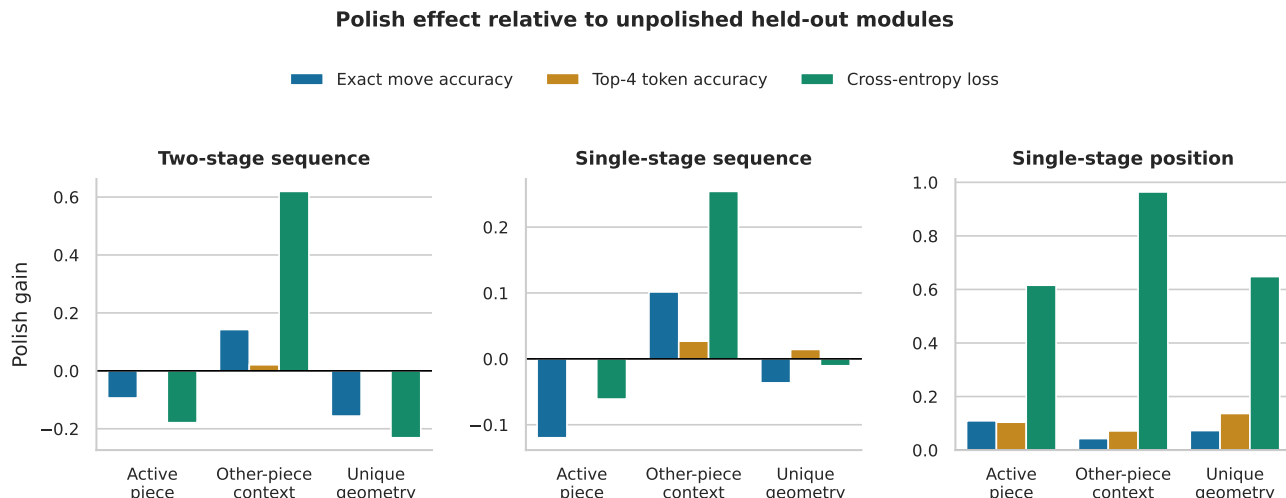


Figure 4: Effect of polishing relative to unpolished held-out modules. The polish pass strongly improves held-out context performance, but can trade off against exact accuracy on held-out active-piece moves in the sequence models.

unpolished evaluation shows that a short polish pass helps to reduce this problem. For the held-out other-piece context subset, a short polish pass reduces loss from 1.02 to 0.77 for the single-stage sequence model, from 1.32 to 0.70 for the two-stage sequence model, and from 1.92 to 0.95 for the position model. The same comparison is summarized in Figure 4.

6.3 Wrong modules destroy held-out geometry

The wrong-module stress test seems to give the clearest evidence that the held-out modules contain functionally relevant information. On 1,024 held-out unique moves, correct held-out modules produce nonzero exact accuracy and nonzero from-to geometry accuracy and replacing the moving piece’s module with the standard module or a seen variant module drives exact accuracy to zero for all three model families with from-to geometry accuracy additionally dropping to zero (Figure 5). This result would seem to be difficult to explain if the behavior were stored entirely in shared layers.

6.4 Training dynamics

Training curves show that the models learn the next-move task and that top-4 token accuracy becomes reasonably high even when exact move accuracy remains lower. This should probably be expected as a move is represented by several tokens in sequence. Figure 6 shows representative training curves from the run histories.

7 Discussion

The results would seem to support a limited but meaningful form of modularity where the held-out rule modules seem to be effective as wrong-module selections destroy held-out move geometry, and correct held-out modules improve unique-rule performance. At the same time, the system is not a clean plug-and-play symbolic rule engine as shown with unpolished held-out modules interfering with unrelated moves in held-out-rule games, potentially indicating that rule-specific adaptation interacts with shared context representations and output decoding. The polish pass is important as it shows this interference can be reduced with a reasonably small number of overall tokens while keeping the rest of the model frozen.

The comparison among architectures is also informative as the position-only model is worse in absolute next-move accuracy, but polishing improves its held-out unique and context performance strongly. This is consistent with the idea that removing move history reduces one path by which the model can infer rule context from sequence patterns alone meaning that the sequence models are

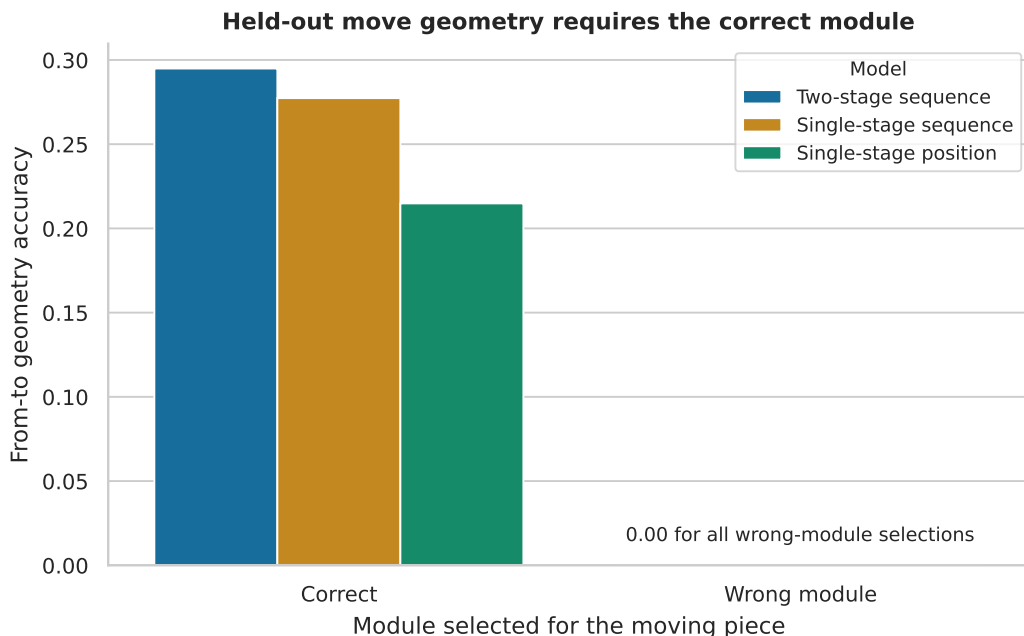


Figure 5: Wrong-module stress test. Selecting the correct held-out module preserves held-out from-to geometry accuracy on K5/N5 unique moves, while selecting a standard or seen-rule module eliminates it.

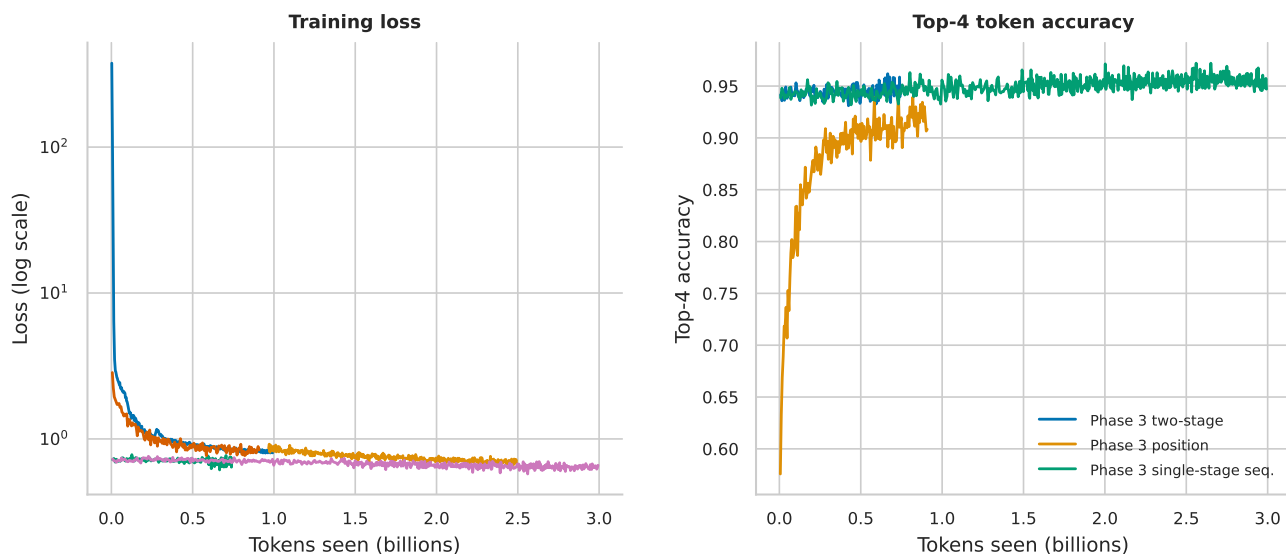


Figure 6: Training dynamics for major runs. Later phases and model variants are not directly controlled comparisons, but the curves document scale, convergence, and throughput.

stronger overall, but they can potentially rely more on contextual regularities outside the modules. The two-stage model has the best ordinary performance among the evaluated models, but its held-out exact accuracy is more sensitive to polishing tradeoffs.

These results suggest that structured rule-family training can induce functional modularity, but the pressure as applied here is incomplete. In domains such as programming languages, where local syntax and semantics vary systematically across related rule systems, a similar approach might help reduce interference during adaptation. Chess variants are far more controlled than language tasks

like using natural software ecosystems, so the current evidence should be interpreted as a proof of concept rather than a general solution.

8 Limitations

This work has several limitations. First, the environment is synthetic and highly structured, chess rule variants are useful for measurement, but they do not capture the ambiguity, distribution shift, or scale of language and code tasks. Second, we report a limited number of training runs rather than statistically powered repetitions across seeds. Third, our module selection is provided by the known ruleset; the model is not required to infer which rule system is active from raw observations alone. Fourth, the held-out adaptation protocol demonstrates partial localization but not perfect noninterference: context interference is measurable and must be reduced by polishing. Finally, the generated data and training pipeline are substantial, and reproducing the full training process requires significant GPU time.

9 Conclusion

Structured rule variation provides a practical way to test whether transformers can be pressured toward modular behavior. In controlled chess-rule families, piece-rule modules acquire functionally relevant behavior: held-out modules improve held-out move prediction, and wrong module selections eliminate unique held-out move geometry. The strongest caution is that modularity is partial, new modules can interfere with unchanged play unless they are polished in context. Overall, these experiments suggest that rule-family training can create measurable modular structure in transformers, while also showing that architectural modularity alone is not enough to guarantee clean isolation.